

Aplicaciones Web II

Analista de Sistemas y Desarrollo de Software

Trabajo Práctico Integrador

Profesor

- Andres Senn

Equipo de desarrollo

- Bonavia Visconti Mateo – 43.133.957
- Ruffa Lucas Tomas – 46.033.791
- Galizzi Marcos – 45.307.659
- Lantieri Lucas – 41.681.985

ÍNDICE

Desarrollo de una plataforma web para la gestión de pedidos 3
1. **Introducción** 3
2. **Equipo de trabajo y roles** 3
3. **Descripción del proyecto** 4
Flujo funcional propuesto *4*
Alcance por etapas *4*
4. **Justificación y propuesta de valor** 4
5. **Objetivos** 5
Objetivo general *5*
Objetivos específicos *5*
6. **Planificación y cronograma** 5
Etapa 1 — Relevamiento y propuesta 5
Etapa 2 — Implementación del servidor Back-End 6
Etapa 3 — Administración del catálogo (CRUD) 6
Etapa 4 — Autenticación y seguridad 6
Etapa 5 — Despliegue a producción (opcional) 7
7. **Metodología de trabajo y herramientas** 7
Gestión de tareas *7*
Control de versiones *7*
8. **Arquitectura y stack tecnológico** 7
9. **Repositorio** 8
10. **Estado del desarrollo — Etapa 2** 8
Tecnologías a utilizar *8*
Endpoints propuestos *8*
Implementación del Front-End con el servidor *8*
Modularización del proyecto *8*
11. **Estado del desarrollo — Etapa 3** 9
Ajustes sobre la Etapa 2 ..... *10*
Estructura MVC del módulo de productos *10*
Interfaz Front-End de administración *10*
Endpoints propuestos *11*

Desarrollo de una plataforma web para la gestión de pedidos

Propuesta de proyecto e informe de avance

1. Introducción

Este documento presenta la propuesta de trabajo y el estado de avance de un proyecto de desarrollo web pensado para un comercio que actualmente gestiona sus ventas mediante un catálogo estático y canales informales de contacto con sus clientes.

El equipo asume el rol de un equipo de desarrollo externo contratado para digitalizar y automatizar ese proceso: partiendo de un sitio institucional ya existente, el objetivo es construir un servidor propio y una capa de datos que conviertan el catálogo en una aplicación transaccional real, capaz de recibir y confirmar pedidos en línea.

El documento se irá actualizando a medida que avance el desarrollo, incorporando en cada etapa el detalle técnico correspondiente y manteniendo siempre la trazabilidad de las decisiones tomadas por el equipo.

2. Equipo de trabajo y roles

El equipo está compuesto por cuatro integrantes. Si bien cada uno tiene un rol principal para ordenar el trabajo, las tareas de desarrollo del servidor se reparten entre todos a lo largo del proyecto, de forma que cada avance quede reflejado en el trabajo conjunto del equipo y no en el de una sola persona.

Integrante	Rol principal	Responsabilidades
Bonavia Visconti Mateo	Coordinación general y Back-End	Coordinación del equipo y de los tiempos de entrega, modelado de la base de datos e implementación de la autenticación de usuarios.
Ruffa Lucas Tomas	Front-End	Adaptación de la interfaz existente a los nuevos datos dinámicos y validación de los flujos de navegación.
Galizzi Marcos	Back-End y Base de Datos	Configuración del servidor, integración con las fuentes de datos, gestión del repositorio y control de versiones.
Lantieri Lucas	Calidad y Documentación	Pruebas de las funcionalidades desarrolladas, documentación técnica y colaboración en el desarrollo del servidor.

3. Descripción del proyecto

El proyecto consiste en evolucionar el sitio web de toma de pedidos de camisetas de fútbol retro y actuales —hasta ahora un catálogo estático— hacia una aplicación web con servidor propio, capaz de servir la información de productos de forma dinámica y de procesar pedidos de punta a punta.

La interfaz conserva la estructura ya validada con el comercio: una presentación institucional, un catálogo organizado por categorías que ahora se alimentará desde una fuente de datos externa, y un formulario final que actúa como checkout para confirmar la orden.

Flujo funcional propuesto

- El cliente ingresa al sitio y accede al catálogo de productos.
- Selecciona una categoría y visualiza los productos disponibles, obtenidos dinámicamente desde el servidor.
- Agrega productos a su pedido y completa el formulario de checkout.
- El pedido queda registrado y disponible para que el comercio lo gestione, reemplazando el intercambio manual por WhatsApp.

Alcance por etapas

El desarrollo se aborda de forma incremental: primero se construye el servidor que sirve el sitio y expone los datos desde una fuente externa; luego se incorporan las operaciones de creación, lectura, actualización y eliminación necesarias para administrar el catálogo; y finalmente se agrega un sistema de autenticación y control de acceso para proteger esas operaciones administrativas.

4. Justificación y propuesta de valor

El catálogo actual presenta la información de forma estática: cualquier cambio de precio, descripción o disponibilidad de stock requiere modificar directamente el código fuente del sitio, lo cual es lento y propenso a errores.

A su vez, la recepción de pedidos depende de canales informales como WhatsApp. Este tipo de flujo manual suele generar demoras en la respuesta, errores de carga por parte de quien toma el pedido y, en horarios de alta demanda, pérdida directa de ventas por falta de capacidad de atención simultánea.

La propuesta de valor del proyecto es centralizar el catálogo en una base de datos administrable y automatizar la recepción de pedidos a través de un flujo web propio, reduciendo la dependencia de la gestión manual y sentando las bases para futuras mejoras (administración propia del catálogo, autenticación de usuarios, panel de gestión de pedidos).

5. Objetivos

Objetivo general

Transformar el sitio de toma de pedidos, actualmente estático, en una aplicación web transaccional con servidor propio, capaz de gestionar dinámicamente el catálogo y de procesar pedidos en línea.

Objetivos específicos

- Conservar la interfaz Front-End ya desarrollada, adaptándola para consumir datos dinámicos.
- Implementar un servidor propio que sirva el sitio y centralice el acceso a los datos del catálogo.
- Conectar el catálogo a una fuente de datos externa, evitando la dependencia de archivos estáticos.
- Incorporar progresivamente operaciones de administración (CRUD) sobre el catálogo.
- Implementar autenticación y control de acceso para proteger las operaciones administrativas.
- Reducir la dependencia de canales informales para la recepción de pedidos.
- Mantener una organización de trabajo ordenada, con control de versiones y seguimiento de tareas por integrante.

6. Planificación y cronograma

TP	A desarrollar	Fecha límite
TP1	Propuesta por escrito	**28/08**
TP2	Servidor para Front-End	**18/09**
TP3	CRUD	**16/10**
TP4	Login y seguridad del CRUD	**06/11**

El proyecto se organiza en cinco etapas incrementales. Cada una agrupa un conjunto de tareas concretas con un responsable asignado, sin perjuicio de que el equipo colabore de forma cruzada cuando es necesario.

Etapa 1 — Relevamiento y propuesta

Estado: Hecho · entrega 28/08

Tarea	Responsable
Definición del equipo y los roles de trabajo	Todo el equipo

Relevamiento del alcance funcional y la justificación del proyecto	Bonavia Visconti Mateo
Redacción de la propuesta de desarrollo	Lantieri Lucas
Selección de las herramientas de gestión y control de versiones	Galizzi Marcos

Etapa 2 — Implementación del servidor Back-End

Estado: Hecho · entrega 18/09

Tarea	Responsable
Configuración del servidor con Node.js y Express	Galizzi Marcos
Conexión del servidor a la fuente de datos externa (MockAPI)	Galizzi Marcos
Organización modular de archivos y carpetas del proyecto	Lantieri Lucas
Definición y prueba de los endpoints iniciales	Bonavia Visconti Mateo
Integración de los nuevos endpoints con el Front-End existente	Ruffa Lucas Tomas
Redacción del informe de avance de esta etapa	Lantieri Lucas

Etapa 3 — Administración del catálogo (CRUD)

Estado: Por hacer · entrega 16/10

Tarea	Responsable
Organización del proyecto con un patrón MVC modular	Galizzi Marcos
Implementación de Router y de los 5 endpoints del CRUD	Galizzi Marcos / Lantieri Lucas
Migración de los datos a una base de datos real (PostgreSQL o MongoDB)	Bonavia Visconti Mateo
Desarrollo de la interfaz Front-End de administración	Ruffa Lucas Tomas
Documentación de la API con Scalar	Lantieri Lucas

Etapa 4 — Autenticación y seguridad

Estado: Por hacer · entrega 06/11

Tarea	Responsable
Persistencia de usuarios registrados en base de datos	Bonavia Visconti Mateo
Hashing de contraseñas	Bonavia Visconti Mateo
Autenticación mediante JWT y cookies	Bonavia Visconti Mateo / Galizzi Marcos

Habilitación justificada de CORS	Galizzi Marcos
Uso de variables de entorno para datos sensibles	Galizzi Marcos
Ampliación de la documentación de la API	Lantieri Lucas

Etapa 5 — Despliegue a producción (opcional)

Estado: a definir según el avance del proyecto

7. Metodología de trabajo y herramientas

Gestión de tareas

El equipo organiza y da seguimiento a las tareas mediante GitHub Projects. El tablero se divide en cinco columnas —Pendiente, En progreso, En revisión, Bloqueada y Finalizada— y cada tarea se asigna a un responsable y se etiqueta según el tipo de trabajo (Back-End, Front-End, documentación o testing), lo que permite visualizar de forma clara el aporte de cada integrante.

Control de versiones

Se utiliza Git como sistema de control de versiones y GitHub como plataforma para alojar el repositorio. Cada nueva funcionalidad se desarrolla en una rama propia; al finalizarla, se abre un pull request que es revisado por al menos otro integrante del equipo antes de fusionarlo a la rama principal.

Esta dinámica busca mantener un historial de cambios ordenado y trazable, además de que las tareas del tablero queden vinculadas a los commits y pull requests correspondientes.

8. Arquitectura y stack tecnológico

El Front-End no se conecta directamente a la fuente de datos externa. Se implementa un servidor propio que actúa como intermediario entre la interfaz y los datos:

Front-End (HTML / CSS / JS) → Servidor propio (Node.js + Express) → Fuente de datos externa (MockAPI)

- El servidor sirve los archivos estáticos del Front-End desarrollado previamente.
- Expone rutas propias que consultan los datos externos y los devuelven al Front-End ya procesados.
- En etapas posteriores, la fuente de datos externa se reemplaza por una base de datos propia (PostgreSQL o MongoDB), sin que el Front-End deba modificarse.

9. Repositorio

El repositorio del proyecto contiene, y se actualiza de forma incremental en cada etapa:

- Código fuente del proyecto.
- Este documento, con la propuesta original y los informes de avance de cada etapa.
- Descripción de los endpoints desarrollados, con su método y una breve referencia.
- A partir de la etapa de administración del catálogo, el archivo SQL o de MongoDB es necesario para replicar la base de datos.

10. Estado del desarrollo — Etapa 2

Esta etapa se encuentra en curso. A continuación se detalla la propuesta técnica sobre la que se está trabajando; esta sección se irá completando con el detalle final a medida que avance la implementación.

Tecnologías a utilizar

- Node.js como entorno de ejecución del servidor.
- Express.js como framework para definir el servidor y sus rutas.
- MockAPI como fuente de datos externa para categorías y productos.

Endpoints propuestos

En línea con el catálogo definido en la propuesta original (liga y productos), se plantean los siguientes dos endpoints de lectura:

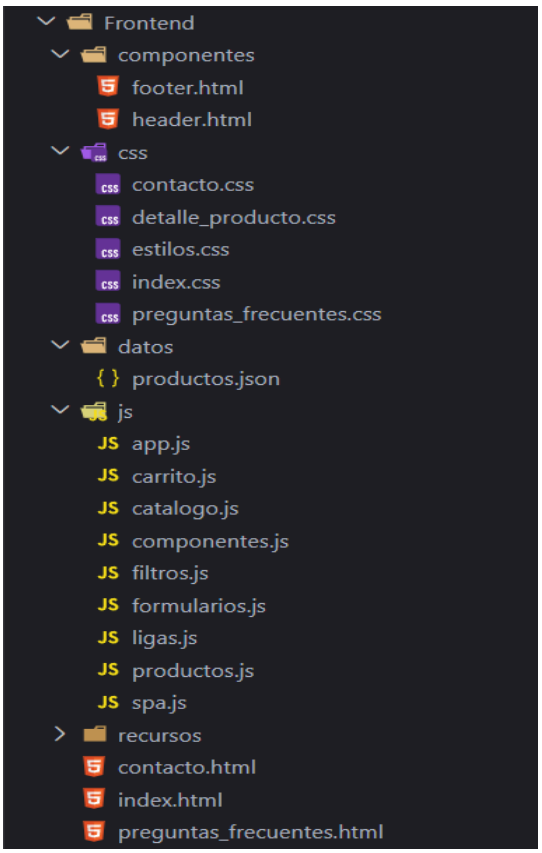
Método	Ruta	Descripción
GET	(base-url)/liga	Obtiene todas las ligas disponibles para filtrar los productos.
GET	(base-url)/productos	Obtiene todos los productos disponibles.

Implementación del Front-End con el servidor

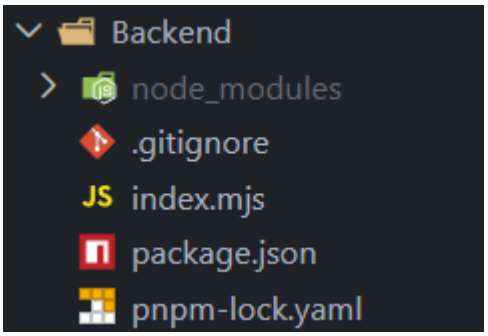
Se modificaron los archivos '**productos.js**' y '**liga.js**' para que, en lugar de leer productos.json de forma local, consuman los recursos que les provee el servidor. Cuando se carga la página web, se solicitan las ligas y los productos a los respectivos endpoints para mostrarlos por pantalla y que el usuario pueda filtrar según lo seleccionado.

Modularización del proyecto

El proyecto se compone de dos carpetas principales: '**Frontend**' y '**Backend**'



Frontend se constituye de las carpetas componentes (elementos reutilizados en todas las secciones de la página), css (estilos), datos (antigua fuente de obtención de datos de los productos), js (funcionalidad general de la web), recursos (elementos gráficos) y tres archivos .html que componen las secciones iniciales de la app.



Backend incluye los archivos necesarios para iniciar el servidor y gestionar las peticiones a los endpoints.

11. Estado del desarrollo — Etapa 3

Esta etapa se encuentra en curso. A continuación se detallan los ajustes realizados sobre la etapa anterior y los primeros avances en la administración del catálogo; esta sección se irá completando a medida que avance la implementación.

Ajustes sobre la Etapa 2

A partir de las observaciones recibidas en la entrega anterior, se realizaron las siguientes modificaciones:

- Se configuró Express para servir los archivos estáticos de la carpeta '**Frontend**', de modo que el sitio completo se accede desde el mismo servidor que expone los endpoints.
- En '**productos.js**' y '**ligas.js**' se reemplazaron las direcciones absolutas (http://localhost:3000/...) por rutas relativas (/productos y /liga). Al compartir el mismo origen, la interfaz ya no depende del host ni del puerto en el que se ejecute la aplicación.
- En '**index.mjs**' se agruparon al inicio las direcciones de la fuente de datos externa (MockAPI), centralizando su configuración.
- Se incorporó un middleware de manejo de errores que registra en consola los errores no controlados y responde al cliente con código 500 y un mensaje en formato JSON.

Estructura MVC del módulo de productos

Para organizar el proyecto bajo el patrón MVC, se creó dentro de '**Backend**' la carpeta '**modulos/productos**' con los siguientes archivos:

Archivo	Responsabilidad
productos.modelo.mjs	Acceso a los datos de los productos.
productos.controlador.mjs	Lógica que procesa cada petición.
productos.vista.mjs	Formato de las respuestas enviadas al cliente.
productos.rutas.mjs	Definición de las rutas del módulo mediante Router.

Por el momento se encuentra creada la estructura; la lógica que hoy reside en '**index.mjs**' se trasladará a estos archivos durante la implementación de los endpoints del CRUD.

Interfaz Front-End de administración

Se desarrolló la interfaz desde la cual se administrará el catálogo, integrada en la página principal:

- Un botón "Crear producto", ubicado sobre el listado, que abre un formulario para dar de alta un nuevo producto.
- En cada tarjeta del catálogo, dos botones para editar y eliminar el producto correspondiente.
- Tres ventanas de diálogo: confirmación de eliminación, edición de producto (con los datos actuales precargados) y creación de producto. Los formularios incluyen nombre, precio, descripción, etiqueta, año e imágenes.

Esta funcionalidad se reúne en el nuevo archivo '**admin.js**', que se inicializa desde '**app.js**'

junto con el resto del sitio; los estilos se agregaron en '**index.css**' y '**estilos.css**'. Actualmente estas acciones son solo visuales: al confirmar, los diálogos se cierran sin enviar datos. El siguiente paso es conectarlas con los endpoints del servidor.

Endpoints propuestos

Para completar la administración del catálogo, se plantean los siguientes endpoints sobre el recurso productos:

Método	Ruta	Descripción
GET	(base-url)/productos	Obtiene todos los productos disponibles.
GET	(base-url)/productos/:id	Obtiene un producto según su id.
POST	(base-url)/productos	Crea un nuevo producto.
PUT	(base-url)/productos/:id	Modifica los datos de un producto existente.
DELETE	(base-url)/productos/:id	Elimina un producto del catálogo.